Lab 1：Data Lab 实验报告

一、各函数的实现思路

1. signMask

这个函数要生成只有最高位为 1 的掩码。整数 1 的最低位为 1，将它左移 31 位，就得到 0x80000000。按照实验规定的 32 位补码模型，这个位模式表示最小负整数。这样只需一次左移，也不用写超过常量限制的大数。

2. bitXor

异或要求两个对应位不同时返回 1，相同时返回 0。本题只能用取反和按位与，所以利用德摩根定律，把“至少一个位为 1”改写为 ~(~x & ~y)，再与 ~(x & y) 相与，排除两个位同时为 1 的情况。这就得到与异或相同的结果。

3. negativePart

我用 ~x + 1 计算补码的相反数，再用 x >> 31 生成符号掩码。算术右移后，负数得到全 1，非负数得到全 0。两者按位与，便能在负数时保留相反数，在非负数时得到 0。最小负整数取负后的位模式不变，按实验的补码规则处理。

4. copyByteWithin

字节编号从最低字节开始，将编号左移 3 位就得到对应的位移量。我先把源字节右移到最低位置，用 255 保留这 8 位，再用目标字节掩码清空目标位置。最后将源字节左移到目标位置并按位或合并。源字节不会被清除，其他字节也保持原值。

5. logicalShift

负数进行算术右移时，高位会补 1，因此我在右移后用掩码清除这些位。掩码由最高位的 1 经右移、左移一位和取反得到，只保留右移结果中有效的低位。n 为 0 时掩码全为 1，n 为 31 时只保留最低位，整个过程没有出现移位 32 位的情况。

6. swapNibblePairs

每个字节包含高、低两个四位组。我用小常量 15 拼出 0x0F0F0F0F，选出各字节的低四位，再把它们左移 4 位。高四位右移 4 位后也用这个掩码保留。将两部分按位或，就完成每个字节内部的交换，例如 0xAB 变成 0xBA，字节顺序不变。

一、各函数的实现思路（续）

7. secondLowestZeroBit

x + 1 会将末尾连续的 1 清零，并把紧邻它们的第一个 0 变成 1。将它与原数按位或，便只填上了最低的零位。设这个结果为 y，再计算 ~y & (y + 1)，就能单独标出 y 中最低的零位，也就是原数的第二个最低零位。不足两个零位时结果为 0。

8. oddParity

这个函数返回需要补上的奇校验位：原数有偶数个 1 时返回 1，有奇数个 1 时返回 0。我依次将 x 与右移 16、8、4、2、1 位的结果异或，把全部 32 位的奇偶性折叠到最低位。最后取最低位并逻辑取反，使补上返回值后，1 的总数成为奇数。

9. rotateRightBits

循环右移需要将右侧移出的位放回左侧。我先用 n & 31 得到实际位移量，再把结果拆成逻辑右移部分和左移绕回部分，最后按位或。右移部分用掩码清除补入的符号位，左移量用补码取负后与 31 相与得到。这样位移量为 0 时也能返回原数，避免移位 32 位。

10. roundEvenPow2

我求出 x 除以 2 的 n 次方后的商，并检查商的奇偶性。给 x 加上“半步长减一，再加商的最低位”的偏置，随后右移 n 位再左移 n 位，清除低位。小于半步时舍去，大于半步时进位；恰好半步时，偶数商不变，奇数商进位，使中点情况选择偶数商。

11. midpointTowardFirst

直接计算 x + y 可能溢出，因此我利用 x + y = 2(x & y) + (x ^ y)，用 (x & y) + ((x ^ y) >> 1) 得到向下取整的中点。若总和为奇数且 x 大于 y，再加 1，使半整数朝第一个参数取整。比较大小时，异号根据符号判断，同号才使用差的符号，避免减法溢出影响结果。

12. isBetweenEitherOrder

我分别判断 x 是否小于 a、是否小于 b。若两次判断不同，x 就位于两个不同端点之间，再补上 x 等于任一端点的情况，便得到包含端点的区间判断。比较时，异号根据符号判断，同号根据差的符号判断，避免溢出。这个方法无需给端点排序；a 和 b 相等时，只有 x 等于该端点才返回 1。

13. mul5Sat

我通过加法依次求出 2x、4x 和 5x，用相邻结果异或后的最高位检查各阶段是否变号。任一阶段出现溢出，就根据原数符号选择饱和值：正方向取 INT_MAX，负方向取 INT_MIN；否则保留 5x。最后用全零或全一掩码选择结果，避免条件分支，也不会漏掉中间溢出后最终符号又恢复的情况。

一、各函数的实现思路（续）

14. classifyAdd3

本题判断三个数最终的精确数学和，而不能只看中间加法是否溢出。我把每个数拆成除以 4 的商和低两位余数，分别相加，再将余数的进位加到商上。得到的 h 等于精确和除以 4 后向下取整，计算过程不会溢出。若 h 的最高三位不全等于符号位，最终和就越界，再按符号返回 1 或 -1，否则返回 0。

15. floatScaleThreeHalves

我拆出符号、阶码和小数字段，NaN 与无穷原样返回。规格化数补上隐藏的最高位，再用整数精确计算有效数的三倍，按大小右移并调整阶码；非规格化数直接处理小数字段。根据丢弃部分与半程的比较进行最近偶数舍入，整个过程只舍入一次。最后处理进位、规格化转换和溢出，并拼回符号。

16. floatRoundEven

返回值仍是浮点编码。我按阶码分类：NaN、无穷和已为整数的数保持原值；绝对值不超过 0.5 时返回带原符号的零，在 0.5 与 1 之间时舍到带符号的 1。其余数根据指数确定小数位数，清除小数位，再比较丢弃部分与半程；恰好半程时根据保留整数的奇偶性决定进位。最后拼回符号，负数舍到零时保留负零。

17. float_i2f

我记录符号，并在 unsigned 中取得绝对值，使最小负整数也能正确处理。通过循环寻找最高的 1，用其位置确定阶码。有效位不超过 24 位时左移补零，否则保留高 24 位，并根据丢弃位进行最近偶数舍入。若舍入产生新的最高位，就右移并增加阶码。最后去掉隐藏位，拼接符号、阶码和小数字段。

18. bitCount

我采用分组累加统计 1 的数量，用小常量拼出三个掩码。先在每两位内计数，再合并为每四位的计数，然后得到每个字节的计数。最后通过右移 8 位、16 位后的加法合并四个字节，并保留低 6 位。因为最多只有 32 个 1，6 位足以表示答案；负数也按完整的 32 位位模式统计。

19. bitReverse

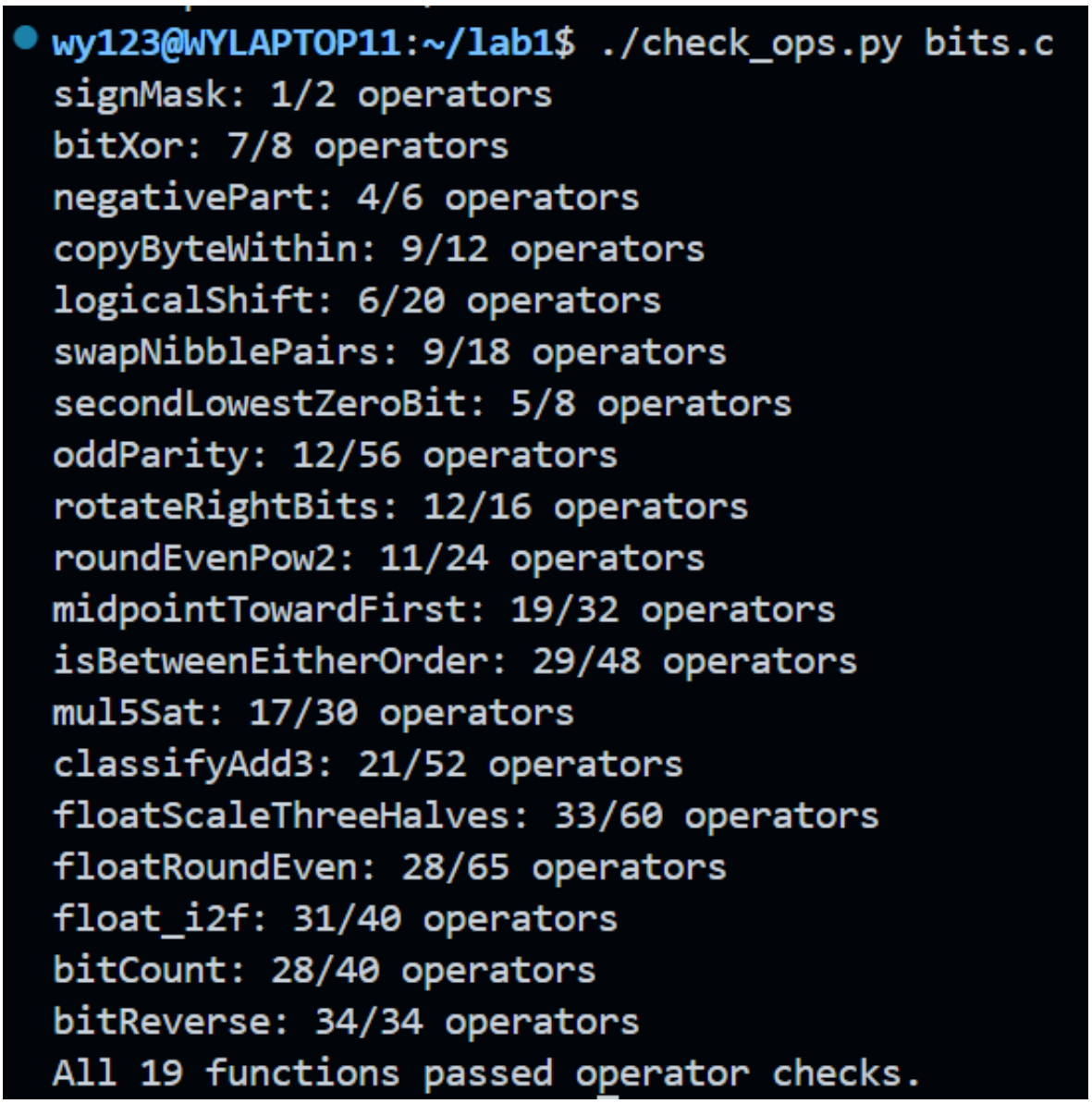
我按 16、8、4、2、1 位的顺序，逐层交换相邻位组，最终让第 i 位移动到第 31-i 位。每层用掩码选出低半组左移，高半组右移后再用掩码保留，两部分按位或合并。掩码从 0x0000FFFF 开始，利用移位和异或逐步更新，减少重复构造的操作，将总运算次数控制在 34 次。

二、运行结果与验证

修改代码后需要重新编译，避免测试旧的可执行文件。运算符检查与正确性测试分别验证代码是否符合限制，以及函数输出是否正确。以下两张图是本次实验提供的实际终端截图。

2.1 运算符与操作数检查

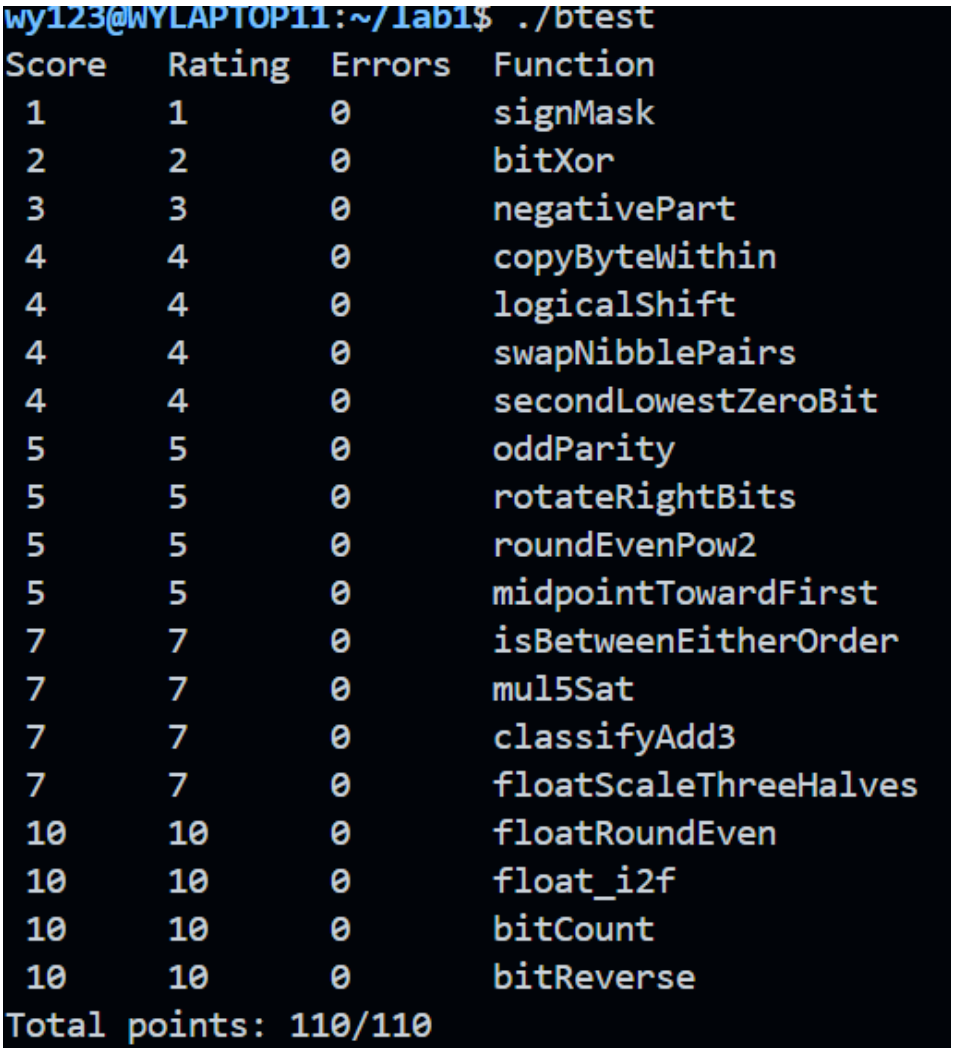
```
./check_ops.py bits.c
```



末行的“All 19 functions passed operator checks.”表明，19 个函数均满足运算符规则和最大操作数限制。

2.2 函数正确性测试

```
./btest
```



测试输出中，19 个函数的 Errors 均为 0，最后一行显示 Total points: 110/110。